Contrôle R5.A.08 — QualDev

Nom/Prénom

```
|
|
|
```

Contrôle

DUT Informatique, IUT de Blagnac, 2023/2024 - 22/01/2024

Conditions

Durée : 1h00 - **Pas de documents** : seule feuille A4 R-V manuscrite non photocopiée.

Barème

1.1) 3 pts - 1.2) 3 pts - 1.3) 2 pts - 2.1) 3 pts - 2.2) 2 pts - 3.1) 3 pts - 3.2) 4 pts - Total) 20 pts - (modifiable pour le bien de tous)

1. Questions de cours

1.1. Concepts importants

Placez les concepts suivants dans un diagramme de classe UML qui permette de comprendre les relations entre ces concepts : Exigence, *User Story*, *Release*, *Issue*, Tâche, Code, Test, Documentation.



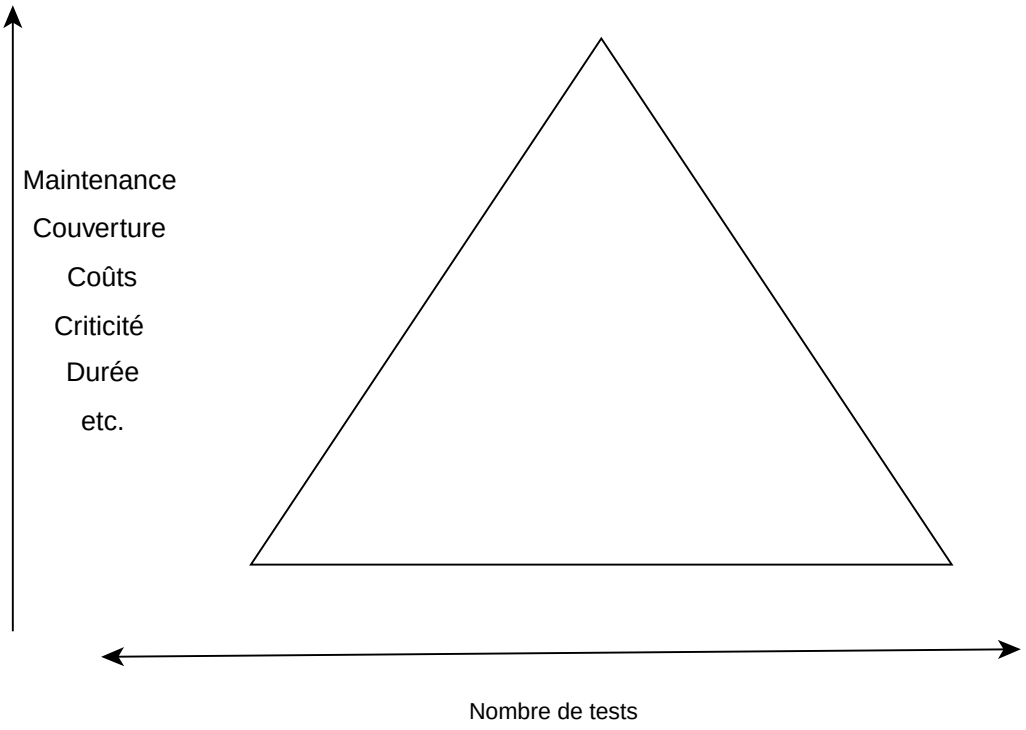
Ne traitez que les concepts que vous comprenez.

Réponse Q. 1.1

```
|
|
|
|
|
|
|
|
|
|
|
|
|
|
|
|
```

1.2. Types de tests

Placez dans la pyramide suivante les types de tests que vous connaissez.



1.3. Concepts importants (suite)

Entre quels concepts (et avec quelles associations/relations) placeriez-vous le concept BDD de *Feature* ?



Ne refaites pas tout le diagramme.

Réponse Q. 1.3

2. Gherkin

2.1. Syntaxe générique

Proposez une meilleure formulation des steps suivants (en 1 seul step par exemple) :

```
Scenario: Bad steps definition
    Given I go to the home page
    Given I check the about page of the website
    Given I get the contact details
```

Réponse Q. 2.1

```
|
|
|
|
|
|
|
|
|
|
```

2.2. Syntaxe avec exemple

Proposez une suite à cette *feature* :

```
Scenario Outline: Eating
    Given There are <start> cucumbers
    When I eat <eat> cucumbers
    Then I should have <left> cucumbers
|
|
|
|
|
|
|
|
|
|
```

3. BDD en Java

Soit la classe Java suivante, proposez une feature simple et l'implémentation de ses *steps*.

```
class Order {
    private String owner;
    private String target;
    private List<String> cocktails;
    private String message;

    public Order() {
        this.cocktails = new ArrayList<>();
    }

    public void declareOwner(String owner) {
        this.owner = owner;
    }

    public void declareTarget(String target) {
        this.target = target;
    }

    public List<String> getCocktails() {
        return cocktails;
    }

    public void addCocktail(String cocktailName) {
        cocktails.add(cocktailName);
    }

    public void nbrCocktail(int numberOfCocktails) {
        this.cocktails.clear(); // Clear existing cocktails if you only want the
number specified
        for (int i = 0; i < numberOfCocktails; i++) {
            addCocktail("cocktail " + (i + 1)); // Add a unique name to each cocktail
        }
    }

    public void addMessage(String message) {
        this.message = message;
    }

    public String getTicketMessage() {
        return "From " + owner + " to " + target + ": " + message;
    }
}
```

Réponse Q. 3-Feature

```
|
|
|
|
|
|
|
|
|
|
|
|
|
|
```

Réponse Q. 3-Steps

```
|
|
|
|
|
|
|
|
|
|
|
|
|
|
```